

Learning L-system Models of Plants Matching Real Images Using Genetic Algorithm

Nazifa Azam Khan¹[0000–0003–2606–6078], William Bushell¹[0009–0008–5412–9930],
Mikolaj Cieslak²[0000–0003–2138–6865], and Ian McQuillan¹[0000–0002–7998–4430]

¹ University of Saskatchewan, Saskatoon, SK, Canada.

{nazifa.khan, william.bushell}@usask.ca, mcquillan@cs.usask.ca

² University of Calgary, Calgary, AB, Canada.

msciesla@ucalgary.ca

Abstract. Procedural plant models provide a mathematical formalism to model plant growth. Lindenmayer systems (L-systems) provide one such formal system, and they have been used to create visually-accurate simulations in different environments. Here, we provide a novel algorithm and implementation for another application of procedural models. Given an L-system and a sequence of real images of a plant developing as input, we use a genetic algorithm to search the parameters of the L-system, where the resulting simulations are geometrically matched to the real images. This matching uses a new graph-theoretic distance function. This work allows for using L-systems directly with genetic algorithm for phenotyping tasks rather than using generalized models such as artificial neural networks. Indeed, a simulation that best matches real images can be used directly for predictions on other real images. A Docker implementation is available for easy deployment.

Keywords: L-systems · Synthetic plants · Plant phenotyping · ANNs · Genetic algorithm.

1 Introduction

Automation in agriculture is a vital step for ensuring food security [20]. Acceleration is needed in crop breeding and agriculture management to feed the world’s growing population, and to adapt to climate change. Precise and efficient measurement of plant traits is important for advancing crop breeding programs and understanding plant development [4]. Plant phenotyping refers to the recording of growth, development, diseases, stress response, and yield; and to understanding how these traits are influenced by its genotype and environment. Manual inspection of plant phenotype is not only time consuming and laboursome, but also prone to human error [34]. Therefore, researchers have turned to automated methods based on the acquisition of a large number of images.

Advanced computer vision, image processing, and machine learning techniques can be applied on acquired images to extract key phenotypic traits. Artificial neural networks (ANNs) and deep learning are providing excellent results on

many data analysis, image analysis, and prediction tasks [13], and have successfully been used in image classification, object identification, and segmentation [26]. A comprehensive summary of deep learning algorithms for identification, regression, prediction, and identification of plant stress phenotypes can be found in survey paper [35]. One challenge of applying ANNs for plant phenotyping problems using supervised learning is obtaining a large amount of labelled training data. Image datasets of the desired plant species, environment, scale, and size, with labelled phenotypic properties of interest, are most likely not available, due to the difficulty associated with collecting and annotating this type of data [36].

Procedural plant models are another important area that uses mathematical formalisms to model plant growth and to create simulations of plant development. Lindenmayer systems (L-systems), introduced by Lindenmayer in 1968 [27] are one of the most common types of models used for this purpose. They are a string rewriting system that uses parallelism to rewrite all letters of a string, and this iterates to produce a sequence of strings. The complicated and repetitive structure of plants make L-systems an excellent method for modelling and simulating them [28]. In graphical L-systems, an interpretation of certain letters is associated to an instruction within a simulator. Where each string can describe an image, a sequence of strings can describe a sequence of images to simulate a temporal process, that can be visually simulated by software such as the “virtual laboratory” (vlab) [32]. With graphical applications of L-systems, it is possible to represent plant structures such as branches, leaves, and other organs [33]. A number of different types of L-systems have been studied [29]. Parametric L-systems are one such type that allow explicit inclusion of growth functions into L-system specifications, where formal parameters are used in the specifications of the productions [28]. The simulations from these types of L-systems can incorporate different plant geometries [28], environmental factors [3], and mechanistic controls [30].

One application of procedural models towards phenotyping is to use synthetic images, that are implicitly labelled, to train neural networks. There are examples of them being used either alone or in conjunction with real images to improve prediction, thereby reducing the cost and labor of image labelling. Synthetic training data was used in the following applications: for early disease detector on tomato plants [24], for training a deep learning network for leaf counting in rosette plants of *Arabidopsis thaliana* [36], for training a deep learning model that detects artichoke seedlings in images captured by a robot [15], for generating agricultural scenes focusing on soybean crops along with various weeds and augmenting them with real world training data to identify crops and weeds [10], for obtaining better results for sweet pepper fruit detection [1], and in some other agricultural examples as described in [2]. It was shown that the level of realism in the synthetic images can have a substantial effect on the predictive results [22] using neural networks. They also showed how L-system models can be calibrated and refined manually using visual comparison, requiring expertise to identify the dissimilarities between the real and synthetic images. This is not

only time-consuming, but also prone to human error, and requires significant expertise. An alternative approach to this is implementing an automated approach for model calibration.

Evolutionary algorithms are natural to explore the search space of L-systems in a variety of ways. In [21], the authors demonstrated the application of evolutionary methods for L-system inference, which is to determine an appropriate axiom and rewriting rules to model a given structure formation or growth process. Ochoa [31] use genetic algorithms and L-systems to simulate the evolution of 2D plant morphologies. Curry implemented a user interface for creating parametric L-systems for people who are not L-system experts [12], by allowing users to guide the evolution of the using genetic algorithm techniques. Hornby et al. [19] described a system that uses an evolutionary algorithm to evolve parametric L-systems. In [16] they use genetic algorithm to evolve L-systems, with the genome consisting of the production rules and variables. Another work used genetic algorithm for the evolution of both rules and interpreter of L-systems to generate a virtual landscape [5]. Bernard et al. [8] inferred a context free L-system from a given sequence of generated strings using a genetic algorithm (extended to parametric L-systems [7]). Inferring different aspects of L-systems is natural with genetic algorithms due to the discrete string representation of the systems.

In our study, we propose a novel technique to automatically find the (close to) best simulation (and hence synthetic images) of an L-system by tuning its parameters to match a temporal sequence of real images, with an evolutionary algorithm. This opens up the possibility of automated approaches for creating accurate simulations, and for using L-systems directly for phenotyping tasks rather than using generalized models such as artificial neural networks. We use an existing parametric L-system model for maize, and we use a genetic algorithm to search the parameter space of the L-system, with a type of geometric matching of the corresponding temporal simulation to a sequence of real images of a plant development. This involves the creation of a novel distance function that quantifies how far a temporal plant simulation is from a real sequence of images of a plant developing (or between two plants). This distance is calculated by first creating a weighted, undirected, bipartite graph between each pair of corresponding real and synthetic images, by using the Euclidean distance between pairwise components of both plants. It then uses the Hungarian algorithm, a matching algorithm from graph theory (also known as the Munkres algorithm) to determine the mathematically minimal sum of weights of any one-to-one mapping between the two given sets of vertices [25]. By performing this across corresponding images of the sequences, and by then adding the matched weights, we can quantify the difference between the two image sequences, which forms the distance function. Note that, in this study we did not use any image processing or computer vision techniques. Instead, we use the given ground truth labelling on the real images, and the implicitly known labelling for the simulated images to know the coordinates. If the ground truth had not been available, image processing or computer vision techniques could be used to first infer them.

We use real images of maize obtained from an open dataset [37]. The results showed that this technique can be used to create more geometrically accurate synthetic maize plants, and synthetic plants that matched more closely with given real plants. Images showed how this method helped the procedural model to correct the topology, and refine the geometry.

This is helpful because accurate simulations can be created automatically to demonstrate plants in new environments. It also opens up the possibility of using this method for the purposes of phenotyping and making predictions directly by finding the best simulation, and then using the synthetic images annotating it as phenotype prediction. Hence, L-systems created to model plant development can be used directly for making predictions rather than using generalized models such as ANNs. This is significant as L-systems can model visual [28], genetic [30], and environmental [3] information to influence phenotyping.

2 Data

A dataset of real maize plant images was used, as well as an L-system that can generate synthetic images.

Real Maize Plants: Real images of maize were obtained from an open dataset created at the University of Nebraska-Lincoln [37]. The dataset (called UNL-CPPD-I) has images of 13 different maize plants of different genotypes. Plants were imaged once per day for 27 days (one plant for only 26) using the visible light camera of the UNL Lemnatec Scanalyzer 3D high-throughput phenotyping facility [9]. Images were taken from two different side views separated by 90 degrees. The dataset also contains ground truth annotations of both the leaf apex and base for each leaf that is visible. Leaves were only annotated in the ground truth if they were visible in that specific image, and were not completely occluded, which is apparent as the number of annotated leaves from the two views differs noticeably. While this approach is beneficial for identifying leaves within an individual images, it does impede the evaluation of leaf identification procedures that try to identify leaves even if they are occluded, which is especially interesting to know. Additional details regarding the imaging setup, dataset organization, ground truth annotation strategies, and genotypes can be found in [9].

Maize plants follow an alternate phyllotaxy, which means that each leaf emerges on the opposite side of the previous leaf, and therefore the leaves form a planar-like surface. A preprocessing step was used to determine the best view between the two available views which is the same procedure as described in [6]. Briefly, the foreground was extracted at first to remove the fixed background of the phenotyping system with a background subtraction method. Then, Otsu thresholding was applied to the grayscale image of the foreground image to obtain the segmented image, and another thresholding was performed by calculating the excess green index of the segmented image. Finally, the convex hull area of the binarized plant images of both views were calculated for each plant

and each day. The view with the larger convex hull area was then selected as the view for that plant/day.

Only the resulting images chosen via this procedure, and their given ground truth annotations were used for all subsequent analyses. Lastly, only images with at least one leaf were kept for each maize plant, and this dataset is known as the real maize dataset. The number of images of each plant varies, as the plant growth rate is different for different genotypes. However, most plants started to show leaves from day 2; hence the total number of images in each varies from 23 to 26. In total, the dataset contains 328 images.

Synthetic Maize Plants Cieslak et al. [11] presented an interactive method for creating and calibrating developmental plant models for maize using parametric L-systems. The model was specified in the L+C modelling language integrated into vlab. Here, we use this L-system of maize as a starting point.

3 Method

We will first describe the fitness function which is used between a sequence of images of a real plant $\mathbf{x} = (x_1, \dots, x_m)$ and a sequence of synthetic images $\mathbf{y} = (y_1, \dots, y_m)$ produced from a simulation. We first describe the method that can be used generally for any sequence of real with any sequence of simulated images, and any type of phenotype. After explaining, we will describe the specific dataset and phenotype that on which it is evaluated. We use the number of real images as m here and match the number of simulated images to be the same number. The image x_1 is compared to y_1 , x_2 to y_2 , etc. until x_m is compared to y_m . For each i from 1 to m , the components of the same type (e.g. leaves or any other property) of x_i are compared to those of y_i as follows. For x_i , it uses the given labelling from the real plant images. For y_i , these values are known exactly since they are simulated. The Euclidian distance between each component of x_i with each component of y_i of the same type is calculated. This produces a weighted undirected bipartite graph where the components of x_i are one set of vertices, those of y_i are the other set, edges are placed between each component of x_i with each component of the same type in y_i , and the Euclidian distance between their positions in 2D are the weights on the edges between each pair (essentially representing the distance between the two components if the two images were over laid together). For each i , the Hungarian matching algorithm [25] is then used between vertices of the two sets (of x_i and y_i) to find the best match. This algorithm produces the mathematically minimal matching between components of the corresponding two sets of vertices of the bipartite graph, and also does so in polynomial time. The sum of the weights of the matched edges produces a distance between these two fixed images. This is added across all m days to produce a distance between real plant $\mathbf{x}$ and synthetic plant $\mathbf{y}$. This distance is used as the fitness function for the genetic algorithm.

For this particular open dataset, the only type of annotated components are leaves, which have both their base and apex annotated together. These annotated values are specific pixel coordinates on the real images, and the inferred

coordinates on the simulated images. So, in this case, if $\mathbf{y} = (y_1, y_2, \dots, y_{27})$ is a synthetic plant, and $\mathbf{x} = (x_1, x_2, \dots, x_{27})$ is real plant, then y_i and x_i represents the i^{th} day image of synthetic plant $\mathbf{y}$ and real plant $\mathbf{x}$ respectively. The distance between $\mathbf{y}$ and $\mathbf{x}$ is defined to be $c = c_1 + c_2 + \dots + c_{27}$, and c_i is the distance between y_i and x_i calculated for day i . To calculate c_i , let $r_1, r_2, \dots, r_n$ be the leaves in x_i , and $s_1, s_2, \dots, s_k$ be the leaves in y_i . We first map each leaf of the real plant image with the synthetic plant image by the Hungarian algorithm. The Euclidian distance between both the base and apex are calculated. Hence, if the base and apex of a synthetic leaf is (u_b, v_b) and (u_a, v_a) respectively, and for a real leaf r is (p_b, q_b) and (p_a, q_a) respectively, then their distance is

$$\sqrt{(u_b - p_b)^2 + (v_b - q_b)^2} + \sqrt{(u_a - p_a)^2 + (v_a - q_a)^2}. \quad (1)$$

This way, we calculate the distance of one leaf pair. It then uses the Hungarian algorithm to compute the optimal matching between the two sets. To note, if either of the images has fewer leaves, we penalize the distance by multiplying the difference by the maximum distance among any pair. A fixed penalty was not used so the method would be generalizable to other datasets. Finally, the distance c_i is the sum of the weights on the matched edges plus the penalty, and c is the distance between the image sequences.

The following 12 parameters were used as part of the L-system generation with minimum and maximum values allowable in parantheses:

- maximum number of leaves according to the plant’s age [1, 10],
- production of a new internode and leaf by the apex reaching the threshold age defined by a constant [0.1, 3.0],
- parameters contribute in internode modelling:
 - target length scaling factor [0.05, 0.7],
 - target radius of an internode [0.01, 0.5],
 - target width scaling factor [0.01, 0.9],
- plant posture, e.g., its initial angle away from a vertical axis [0, 4.0],
- the increase in branching angle over time [5.0, 135.0],
- a leaf length scaling factor [1.0, 5.0],
- parameter contributing in the local width of the leaf [0.01, 0.5],
- a parameter controlling leaf width with respect to leaf length [0.01, 0.5],
- leaf bend [3.0, 90.0] and twist [3.0, 180.0] parameter which controls the leaf shape.

More details about these parameters can be found in [11]. Given particular values for these 12 parameters, the L-system simulates deterministically. The L-system was calibrated to use the same timeline as the real plants. However, only a single view was captured for each day and plant, taken directly orthogonal to the leaves. Ground truth annotations for the leaves (leaf apex and base points) were automatically determined from the model. Annotations were calibrated to the real images in that it only counted a leaf if it was not occluded (algorithmically determined). Had the ground truth labels on the real images been annotated to include occluded leaves, we would have not used this assumption.

The genetic algorithm uses the 12 parameters above as its ‘genes’. That is, each gene is represented by a floating point number, and the genome is the sequence of the 12 parameter values. It maintains a population of 100 synthetic plants, with methodology described as follows, which it executes on each real plant $\mathbf{x} = (x_1, \dots, x_m)$ as input.

- Step 1: Let P be the population of synthetic plants. Initialize P by generating 100 random sets of values for its parameters, and creating a simulation from each.
- Step 2: For each synthetic plant $\mathbf{y} = (y_1, \dots, y_m)$ in P , find the distance of $\mathbf{x}$ to $\mathbf{y}$ with the fitness function.
- Step 3: Choose the 50 synthetic maize plants $C \subset P$ with minimum distance to the real plant. Construct two sets C_1 and C_2 where each C_1 and C_2 contains 25 pairs of plants from C , randomly chosen from C , without replacement (so each $\mathbf{y} \in C$ occurs in exactly one pair of C_1 and one pair of C_2).
- Step 4: From the 50 total pairs in $C_1 \cup C_2$, use random crossover to create 50 new pairs of plants. For crossover, two parent’s 12 parameters were taken as two array of real numbers (genome), and then swapping segments of their encoded parameters at one randomly chosen points to produce offspring that inherit traits from both parents. Hence this creates 100 new plants, which we call P' . It then undergoes mutation on P' by randomly selecting one parameter, and adjusting its value within the allowable range randomly. No probability was applied during crossover and mutation. Let P'' be the resulting 100 new synthetic plants. Calculate the fitness function with each plant in P'' to the real plant $\mathbf{x}$.
- Step 5: From the newly generated 100 synthetic plants in P'' and the previously chosen 50 synthetic plants in C , choose the 100 synthetic maize plants with minimum distance with the real maize plant. Set this to P .

We repeat the same process for 1000 generations, and the best synthetic plant with the smallest distance among them is considered the best match. We then re-run this entire genetic algorithm five times to get the five best matching synthetic plants from five random runs.

A Python script was written to complete this method, which runs vlab in a loop to generate the synthetic images and ground truth annotations for each synthetic plant. The L-system code and python code from [11] were modified for this algorithm. Initially, this experiment was done only sequentially on a single core of a CPU. Due to the time consuming nature of the simulation, the method was updated using Docker, with additional optimizations, whereby multiple instances could be run in parallel. The experiment runs inside Docker containers built from an Ubuntu base image. Images are produced from the project Dockerfile, which copies the project directory into the image, and containers are instantiated and orchestrated with docker-compose, and helper Bash scripts. In-container configuration of Vlab is automated so instances can be provisioned and customized programmatically. Docker code link is available in final version.

4 Results

Following the above described method, results were calculated for the thirteen real maize plants of different genotypes. The whole genetic algorithm was repeated five times for each real plant. Table 1 shows the minimum and maximum distance over the 1000 generations for the first run (minimum is best, and maximum is shown as a comparison), with the improvement in percentage over the 1000 generations, and the minimum distance with the corresponding best synthetic plant among the five random runs, and the standard deviation between the minimum distances of the five random runs. Most of these are visualized in a histogram in Figure 1a. Figure 1b shows a comparison between the number of leaves in the minimum-distance plant of the first run, the maximum-distance plant of the first run, and the minimum-distance plant overall.

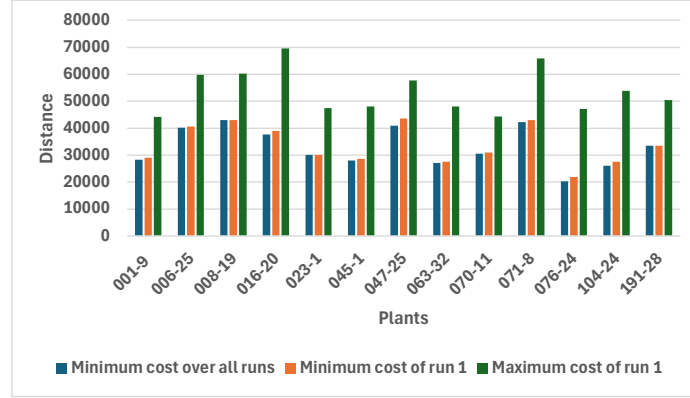
Table 2 shows both real and synthetic images spread across the timeline for one particular plant. Both the minimum and maximum distances in the first run, and the synthetic plant with minimum distance over five random runs are shown. Smaller distances are better. Results for other days of this plant, and for the other twelve plants, are extensive and are omitted due to space constraints, but are provided in Table A1 of [23]. Convergence of the genetic algorithm is mathematically guaranteed as the algorithm keeps a population of best matches in subsequent generations.

5 Discussion and Limitations

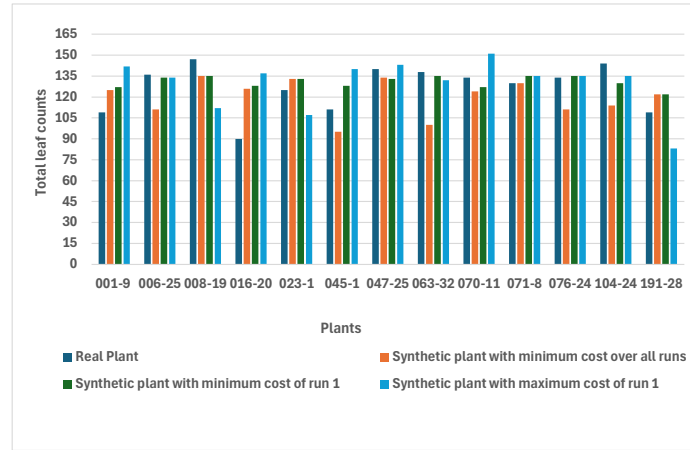
Results for all of the plants showed how the minimized distance was found using the genetic algorithm. Running the whole process five times often showed smaller (better) distance values in comparison to doing only one run. The standard deviations among the minimum distances of the five random runs quantifies the difference between runs, and gives an indication of the benefit of additional runs. It is important to recognize that this minimization is across the entire developmental simulation of the plant, rather than being calibrated to a single image.

Figure 1a visualizes the best vs worst distance for this plant and indicates that the distance is substantially better in the smallest. The distance values on their own in Figure 1a and Table 1 can be difficult on their own to interpret, and they might seem quite large. However, the distance function adds euclidean distances of each component, which are then again added across the sequence. However, the results demonstrate how iterating the generations of the genetic algorithm for parameter tuning reduces the distance between the best matching synthetic plant with the real plant, by an average of 62.72%. Hence, the genetic algorithm is providing substantial and quantitative improvement to the scores vs. using a random simulation. The average standard deviation among the five best synthetic plants is 1.65% of the minimum distance across all runs. Also, the visual assessment of the matching in Table 1 is important.

To note, even the worst sequence of synthetic images can visually look somewhat similar to the real sequence of images. This is because the model was



(a)



(b)

Fig. 1: (a) Histogram showing the distances of the corresponding synthetic plants (lower bars are better). (b) Histogram showing the total number of leaves in the real plant, and in the synthetic plant with minimum distance over all runs, minimum distance over the first run, and maximum distance over the first run. In both of the graphs, the X-axis shows the real plants.

Real Plants Name	Minimum distance in first run	Maximum distance in first run	Improvements in % $(1 - (column2/column3)) * 100$	Minimum distance with the corresponding best synthetic plant among the five random runs (Standard Deviation among the minimum distances of five random runs)
001-9	29013.37 (generation 624)	32176.62	34.4%	28240.57 (345.50)
006-25	40649.61 (generation 340)	59711.10	31.93%	40227.54 (608.43)
008-19	43012.08 (generation 281)	60243.73	28.61%	43012.08 (432.90)
016-20	38973.77 (generation 246)	69616.15	44.02%	37614.38 (1262.01)
023-1	30109.97 (generation 176)	47498.36	36.61%	30109.97 (524.75)
045-1	28613.90 (generation 306)	48090.81	40.51%	27988.57 (321.77)
047-25	43519.08 (generation 409)	57705.34	24.59%	40916.33 (1028.55)
063-32	27494.74 (generation 589)	48089.09	42.83%	27087.52 (267.72)
070-11	30909.65 (generation 139)	44345.17	30.3%	30436.55 (392.13)
071-8	43015.29 (generation 304)	65917.34	34.75%	42282.00 (565.26)
076-24	21873.84 (generation 658)	47081.67	53.55%	20273.68 (643.84)
104-24	27490.91 (generation 814)	53830.66	48.94%	26071.16 (850.62)
191-28	33522.45 (generation 857)	50472.09	33.59%	33522.45 (484.73)

Table 1: The minimum (column 2, with generation) and maximum (column 3) numerical distances for the first run, and the minimum distance across all five runs (column 5) for all the 13 plants labelling the rows. The standard deviation among the best distances of the five runs is presented in the fifth column within first brackets. Lower numbers are better.

created to closely reflect this exact image dataset [11], including being visually calibrated, and following alternate phyllotaxy. In practice, it might not be needed to create such a precise and well calibrated model to a specific dataset when creating an initial L-system. It is likely more important to capture all the

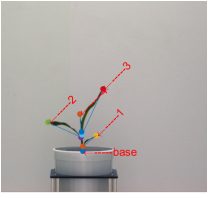
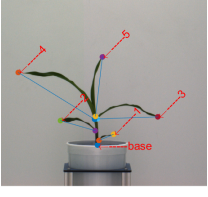
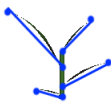
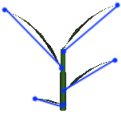
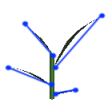
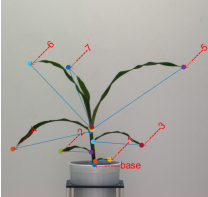
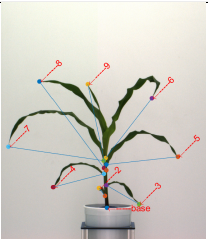
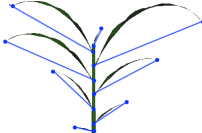
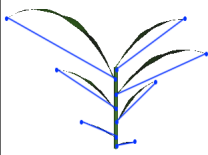
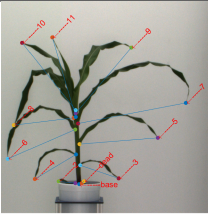
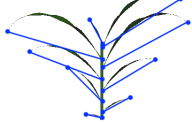
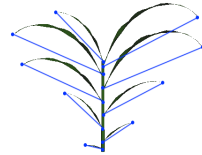
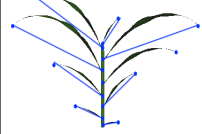
Days	Real Plant Images	The Best Synthetic Plant Images from the First Run	The Worst Synthetic Plant Images from the First Run	The Best Synthetic Plant Images from the Five Random Runs
7				
12				
17				
22				
27				

Table 2: The real and obtained synthetic plant images an interval of five days for Plant 047-25.

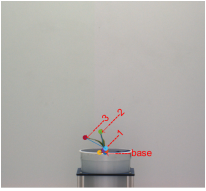
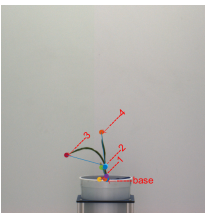
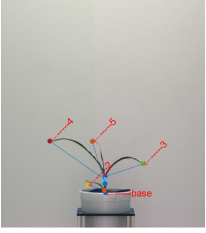
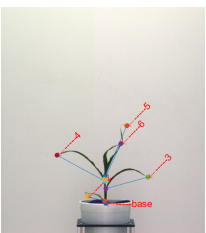
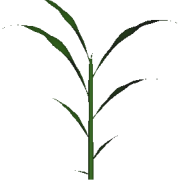
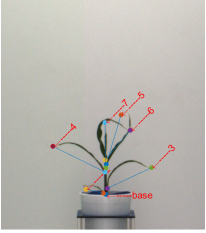
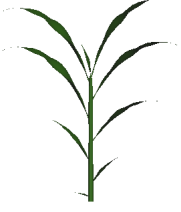
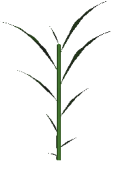
Days	Real Plant Images	The Best Synthetic Plant Images from the First Run	The Worst Synthetic Plant Images from the First Run	The Best Synthetic Plant Images from the Five Random Runs
7				
12				
17				
22				
27				

Table 3: The real and obtained synthetic plant images an interval of five days for Plant 016-20.

variation that is possible in real images within the L-system. Even if the model can generate variations that are not present in any real images, the genetic algorithm can find realistic parameter values. Nevertheless, the precise geometry was quantitatively improved across generations.

The first row of Table 2 shows that the first leaf of the real plant started from the right side of the stem. Both the best and worst synthetic plant from the first run have their first leaf starting from the left side of the stem. However, the best synthetic plant across all runs has corrected this, and the first leaf of this synthetic plant has started from the right (and correct) side of the stem.

In terms of the more crude method of only matching the number of leaves (rather than the precise positions of every leaf), results are shown in Figure 1b. The benefits of multiple runs on obtaining the correct number of leaves is not large. For example, plant 047-25 with the lowest distance among five runs matched the number of leaves with the real plant’s leaf numbers. In contrast, plant 070-11 shows that the plant with the lowest distance over five runs mostly did not match the real number of leaves (Figure 1, Table A7 of [23]). But if we closely examine the plant images of later days in the sequence, we can see that the topology of the third row of plant images matches with the first row of images better than the other two. Hence, this technique is at present better at matching certain aspects of the geometry of the plant, rather than the cruder measure of the number of leaves. Also, because the ground truth labels only mark the base and apex of each leaf, it is only matching these components. Labelling the entire midvein of each leaf and adjusting the distance function accordingly would improve overall geometric matching of leaves. Alternative distance functions could be explored in the future to accommodate this expansion. It is also evident that one of the main challenges with the current procedural maize model is, it did not consider leaf senescence, when certain leaves of a real plant die. Also, labelling and counting occluded leaves would be preferred. For example, in Table 3 on day 17, one leaf is not visible as it is either dead or hidden.

In all, finding the closest synthetic plant and simulation to a real plant is useful in many ways. The current method could be extended to use procedural models for image-based plant phenotyping predictions. For example, instead of using ground truth labels to train an ANN (as previous works have done as described in Section 1), we can instead find the best L-system simulations which can then predict many more phenotypic properties than the labels. We can also use image processing techniques to determine the properties and avoid manual annotations altogether. Simulations can also be used to showcase how the plant could grow in new environments. We can also find model parameters that most closely match different phenotypic properties, e.g., which parameters and simulations most closely match drought or diseased phenotypes.

The initial implementation without Docker and sequentially on a single core of a CPU, was computationally heavy. For one real plant it is generating almost $(27 \text{ days}) * (100 \text{ synthetic plants}) * (1000 \text{ generations}) = 2,700,000$ images. We initially used mac desktop computers running Python scripts that ran simulations within vlab. This task took between 5 and 10 days for the run of each plant.

However, this computational and time complexity was reduced by introducing Docker and running on a server. The Docker setup allows us to launch and synchronize many parallel instances that each provided an isolated, reproducible runtime for the experiment, and optional volume mappings direct experimental outputs back to the host machine for preservation and analysis. Finally, the runtime was again reduced by turning off the animated mode of vlab, and saving every parameter value so that a synthetic plant can be regenerated with animated mode on as needed. This modification reduced the runtime notably, from 7 days to 2 days for the run of each plant, and multiple plants could be run in parallel.

6 Conclusions and Future Works

This work presents a novel method for finding L-system parameters where the resulting simulation closely matches a time-series of real images of a plant developing. This can be seen as an automated method of the manual calibration process used in [11,22]. There has not previously been any other implemented method of calibrating an L-system with a sequence of images. We applied a genetic algorithm for tuning model parameters, with a distance function based on both Euclidian distance and the Hungarian matching algorithm on a bipartite graph created between each pair of a real and a synthetic image in a sequence, and then adding across the sequence. The improvement across generations was measured quantitatively, with a 62.72% improvement in distance across generations. Synthetic image sequences were also visually compared to the real image sequences, and the best synthetic plant was found to closely match the correct geometry.

Implementing procedural models is more cost and labor effective in comparison to collecting and annotating different plant images for every different phenotyping task. Plant models have been used in the literature for simulating plant growth and incorporating mechanistic and environmental effects [28,17,18]. The work has been examined with maize plants and can be employed with other plant species, or other applications of L-systems.

This study opens many avenues for future research. It is only necessary to create a single general L-system model that captures all the phenotypes possible within a species, and then this method can find parameters that closely match the given phenotype. This allows for parameters to be found that target any treatment or diseased phenotypes and to have more precise simulations based on those. In the future, better computational optimization and parallelization would be useful. Also, it would be interesting to feed the resulting closely matching synthetic images as training data to an ANN for phenotyping tasks. Improving procedural plant models to capture more phenomena, such as, leaf senescence, is important, along with midvein positions. Moreover, other distance functions, such as instance masks, or that of [14] would be of interest, along with using different types of ground truth data. Finally, it would be useful to compare the time taken to manually calibrate an L-system vs. doing so automatically.

References

1. Afonso, M., Barth, R., Chauhan, A.: Deep learning based plant part detection in greenhouse settings. In: 12th International Conference of the European Federation for Information Technology in Agriculture, Food and the Environment. pp. 48–53. EFITA (2019)
2. Afonso, M., Giufrida, V.: Synthetic data for computer vision in agriculture. *Frontiers in Plant Science* **14**, 1277073 (2023)
3. Allen, M., Prusinkiewicz, P., DeJong, T.: Using L-systems for modeling source–sink interactions, architecture and physiology of growing trees: The L-PEACH model. *New Phytologist* **166**(3), 869–880 (2005)
4. Araus, J.L., Kefauver, S.C., Zaman-Allah, M., Olsen, M.S., Cairns, J.E.: Translating high-throughput phenotyping into genetic gain. *Trends in Plant Science* **23**(5), 451–466 (2018)
5. Ashlock, D.A., Gent, S.P., Bryden, K.M.: Evolution of L-systems for compact virtual landscape generation. In: 2005 IEEE Congress on Evolutionary Computation. vol. 3, pp. 2760–2767. IEEE (2005)
6. Azam Khan, N., Lyon, O.A., Eramian, M., McQuillan, I.: A novel technique combining image processing, plant development properties, and the Hungarian algorithm, to improve leaf detection in maize. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops. pp. 74–75 (2020)
7. Bernard, J., McQuillan, I.: Inferring temporal parametric L-systems using Cartesian genetic programming. In: 2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI). pp. 580–588 (2020)
8. Bernard, J., McQuillan, I.: Techniques for inferring context-free Lindenmayer systems with genetic algorithm. *Swarm and Evolutionary Computation* **64**, 100893 (2021)
9. Choudhury, S.D., Bashyam, S., Qiu, Y., Samal, A., Awada, T.: Holistic and component plant phenotyping using temporal image sequence. *Plant Methods* **14**(1), 35 (2018)
10. Cieslak, M., Govindarajan, U., Garcia, A., Chandrashekar, A., Hadrich, T., Mendoza-Drosik, A., Michels, D.L., Pirk, S., Fu, C.C., Palubicki, W.: Generating diverse agricultural data for vision-based farming applications. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 5422–5431 (2024)
11. Cieslak, M., Khan, N., Ferraro, P., Soolanayakanahally, R., Robinson, S.J., Parkin, I., McQuillan, I., Prusinkiewicz, P.: L-system models for image-based phenomics: case studies of maize and canola. *in silico Plants* **4**(1), diab039 (2022)
12. Curry, R.: On the evolution of parametric L-systems. Tech. rep. (2000), <https://ucalgary.scholaris.ca/items/e55d5747-7511-4108-8b71-e9b4e0cd853d>
13. Das Choudhury, S., Samal, A., Awada, T.: Leveraging image analysis for high-throughput plant phenotyping. *Frontiers in Plant Science* **10**, 508 (2019)
14. Ferraro, P., Godin, C.: A distance measure between plant architectures. *Annals of Forest Science* **57**(5), 445–461 (2000)
15. Fiestas, E., Ramos, O.E., Prado, S.: RPA and L-system based synthetic data generator for cost-efficient deep learning model training. In: 2021 IEEE 3rd Eurasia Conference on IOT, Communication and Engineering (ECICE). pp. 645–650. IEEE (2021)
16. Fitch, B.G., Parslow, P., Lundqvist, K.Ø.: Evolving complete L-systems: using genetic algorithms for the generation of realistic plants. In: Artificial Life and

- Intelligent Agents: Second International Symposium, ALIA 2016, Birmingham, UK, June 14–15, 2016, Revised Selected Papers 2. pp. 16–23. Springer (2018)
17. Holliday, D.J., Samal, A.: Object recognition using L-system fractals. *Pattern Recognition Letters* **16**(1), 33–42 (1995)
 18. Holliday, D., Peterson, B., Samal, A.: Recognizing plants using stochastic L-systems. In: *Proceedings of IEEE International Conference Image Processing*. vol. 1, pp. 183–187 (1994)
 19. Hornby, G.S., Pollack, J.B.: Evolving L-systems to generate virtual creatures. *Computers & Graphics* **25**(6), 1041–1048 (2001)
 20. Ibrahim, M., Nabeel, M., Raza, M., Hameed, N., Rafiq, R., Zaheer, M., Ain, N., Iftikhar, Z., Ammar, A., Khalid, M., et al.: The role of technology and innovation in enhancing food security. *Journal of Physical, Biomedical and Biological Sciences* **2023**(1), 14–14 (2023)
 21. Jacob, C., et al.: Evolving evolution programs: Genetic programming and L-systems. *Genetic Programming* pp. 107–115 (1996)
 22. Khan, N., Cieslak, M., Eramian, M., McQuillan, I.: Effectiveness of training with procedurally generated synthetic images of crop plants. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. pp. 5451–5461 (2025)
 23. Khan, N.A.: Using procedural models to improve image-based plant phenotyping. Ph.D. thesis, University of Saskatchewan (October 2025)
 24. Klein, J., Waller, R., Pirk, S., Paľubicki, W., Tester, M., Michels, D.L.: Synthetic data at scale: a development model to efficiently leverage machine learning in agriculture. *Frontiers in Plant Science* **15**, 1360113 (2024)
 25. Kuhn, H.W.: The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly* **2**(1-2), 83–97 (1955)
 26. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
 27. Lindenmayer, A.: Mathematical models for cellular interactions in development I. Filaments with one-sided inputs. *Journal of Theoretical Biology* **18**(3), 280–299 (1968)
 28. Lindenmayer, A., Prusinkiewicz, P.: *The Algorithmic Beauty of Plants*. New York: Springer-Verlag (1990)
 29. Lintermann, B., Deussen, O.: Interactive modeling of plants. *IEEE Computer Graphics and Applications* **19**(1), 56–65 (1999)
 30. Nishida, T.: KOL-system simulating almost but not exactly the same development: case of Japanese cypress. *Memoirs of the Faculty of Science, Kyoto University, Series B* **8**(1), 97–122 (1980)
 31. Ochoa, G.: On genetic algorithms and Lindenmayer systems. In: *International Conference on Parallel Problem Solving from Nature*. pp. 335–344. Springer (1998)
 32. Pavol, F., Prusinkiewicz, P.: Virtual laboratory: An interactive software environment for computer graphics. *Computer Graphics International* **242**, 93–100 (1999)
 33. Prusinkiewicz, P., Lindenmayer, A., Hanan, J.: Developmental models of herbaceous plants for computer imagery purposes. *Computer Graphics* **22**(4) (1988)
 34. Sheikh, M., Iqra, F., Ambreen, H., Pravin, K.A., Ikra, M., Chung, Y.S.: Integrating artificial intelligence and high-throughput phenotyping for crop improvement. *Journal of Integrative Agriculture* **23**(6), 1787–1802 (2024)
 35. Singh, A.K., Ganapathysubramanian, B., Sarkar, S., Singh, A.: Deep learning for plant stress phenotyping: trends and future perspectives. *Trends in Plant Science* **23**(10), 883–898 (2018)

36. Ubbens, J., Cieslak, M., Prusinkiewicz, P., Stavness, I.: The use of plant models in deep learning: an application to leaf counting in rosette plants. *Plant Methods* **14**(1), 6 (2018)
37. University of Nebraska-Lincoln Plant Vision Initiative: UNL plant phenotyping datasets (November 2018), <https://plantvision.unl.edu/dataset>